

AIS dev with GitHub Copilot

Exploring how Azure Integration Services
development has changed with using
GitHub Copilot



Agenda for today

1. Introduction & Goals
2. Developer workflow with GitHub Copilot
3. Demo scenario
3. Skills, instructions, prompts and agents
5. Orchestration with subagents and memory
6. GitHub App - Scaling your workflow
7. GitHub Auto and Hydrafusion
8. Questions

Introduction



Cameron McKay

Senior Manager, MNP LLP

Cameron.McKay@mnp.ca

Cameron McKay is a Microsoft Azure MVP with a strong background in web, integration, and business analysis. Cameron is a thought leader in Azure integration services and has spoken at numerous conferences on topics related to Microsoft Azure. Cameron has blended his expertise in software development and business analysis to deliver a variety of enterprise web, integrations and cloud applications using the Microsoft technology stack.

Cameron spends his off-hours as a tabletop board game enthusiast, cooking up delicious meals, watching anime and training his husky dogs.



Goals for today

Leave you with two ideas and impressions...

01

Understanding

Understand the landscape of where GitHub Copilot fits for development in integration workflows, common issues and resolving those challenges.

02

Application

Take what we learn and use it immediately to improve your everyday development workflow with GitHub Copilot

What connects all of these?

Requirements

Refactoring

Testing

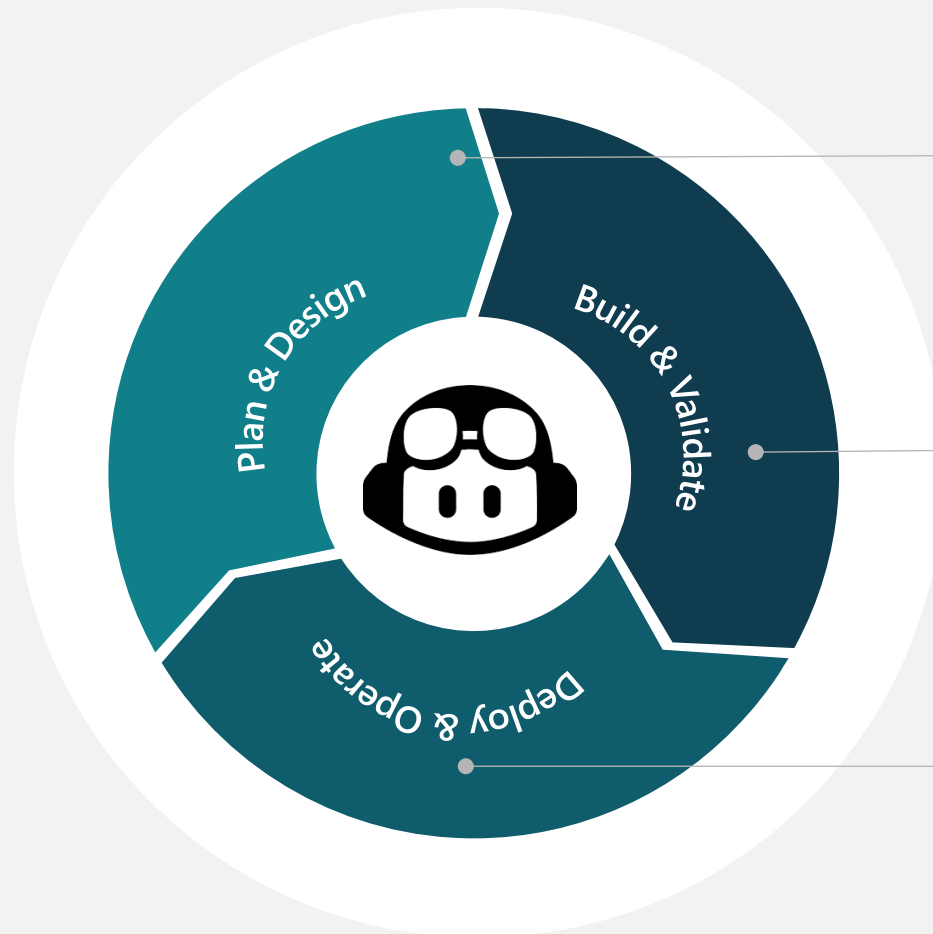
Architecture

Documentation

Development

PR Reviews

GitHub Copilot is everywhere in the Development Lifecycle



Plan & Design

- Requirements
- Specifications
- Architecture
- Planning

Build & Validate

- Coding
- Refactoring
- Debugging & Testing
- PR Reviews

Deploy & Operate

- Deployment
- Monitoring
- Optimization
- Security
- Documentation

Developer Workflow with Copilot

Working with Copilot from end to end



Requirements

Document full requirement and create the prompt to feed into GitHub Copilot



Plan Mode

Feed the requirement and prompt into GitHub Copilot to create an implementation plan.



Implementation

Ask GitHub Copilot to implement the plan and work with it to resolve any issues after local review



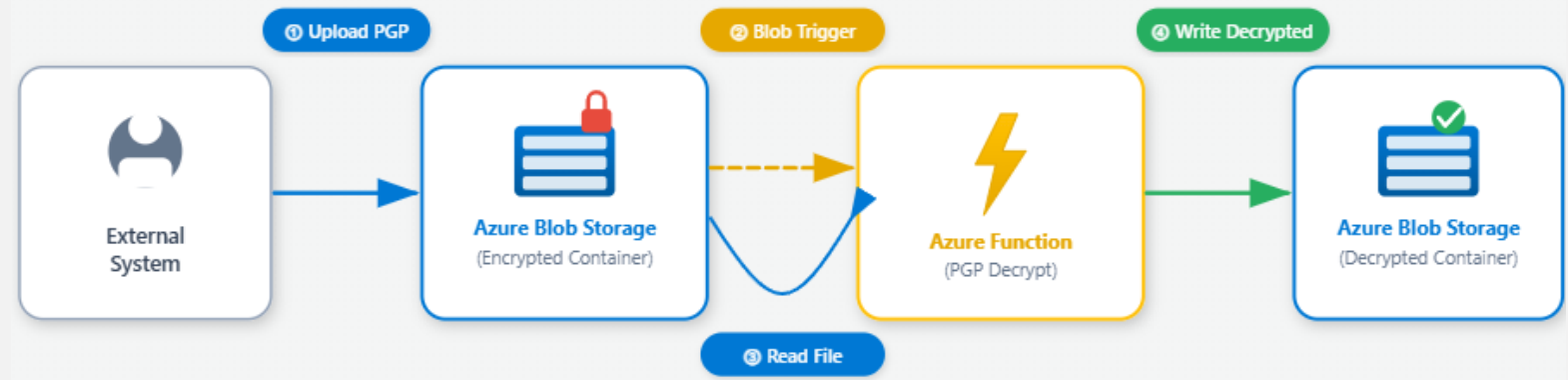
Review and Merge

Pull request creation, agent review, human review, fix issues and merge!

Demo Architecture

PGP Decryption Integration Workflow

We will leverage Azure Function Apps to create a helper endpoint that encrypts and decrypts files in a storage account.



Let's develop with GitHub Copilot

Using GitHub Copilot to develop an integration

What Goes Wrong?

Where AI-assisted development breaks down in practice



Code → Code Quality

- Generated code doesn't follow standards
- Inconsistent patterns and structure
- Hard to maintain long term



Context → Accuracy

- Too much or too little context
- Inconsistent or incorrect outputs
- Difficult to guide reliably



Scale → Workflow

- Limited local workflows (single task focus)
- Difficulty parallelizing work
- Cost and token management

Issue #1 - Code Quality



Problem

Code Quality issues

- Doesn't follow standards
- Inconsistent structure
- Hard to maintain



Reality

What's Happening

- AI lacks context
- No awareness of team conventions
- Generates "generic" solutions



Solution

How We Fix It

- Instructions
- Skills
- MCP

Code Quality Issues

Let's fix them with skills and instructions

Issue #2: Context & Accuracy

AI is only as good as the context it receives

Too Much Context

- Overloaded prompts
- Irrelevant information
- Degraded output quality

Too Little Context

- Missing requirements
- Lack of constraints
- Generic or incorrect solutions



Inconsistent Context

- Different prompts = different outputs
- No shared understanding
- Hard to reproduce results

The Fix: Controlled Context

- Use subagents with focused scope
- Limit what each agent sees
- Use memory intentionally

Context & Accuracy

Let's enhance this with roles and subagents

Issue #3: Scaling AI Development

From single-threaded work → parallel execution

1

Before | Single Workflow

- One local worktree
- One task at a time
- Manual context switching
- Slow iteration



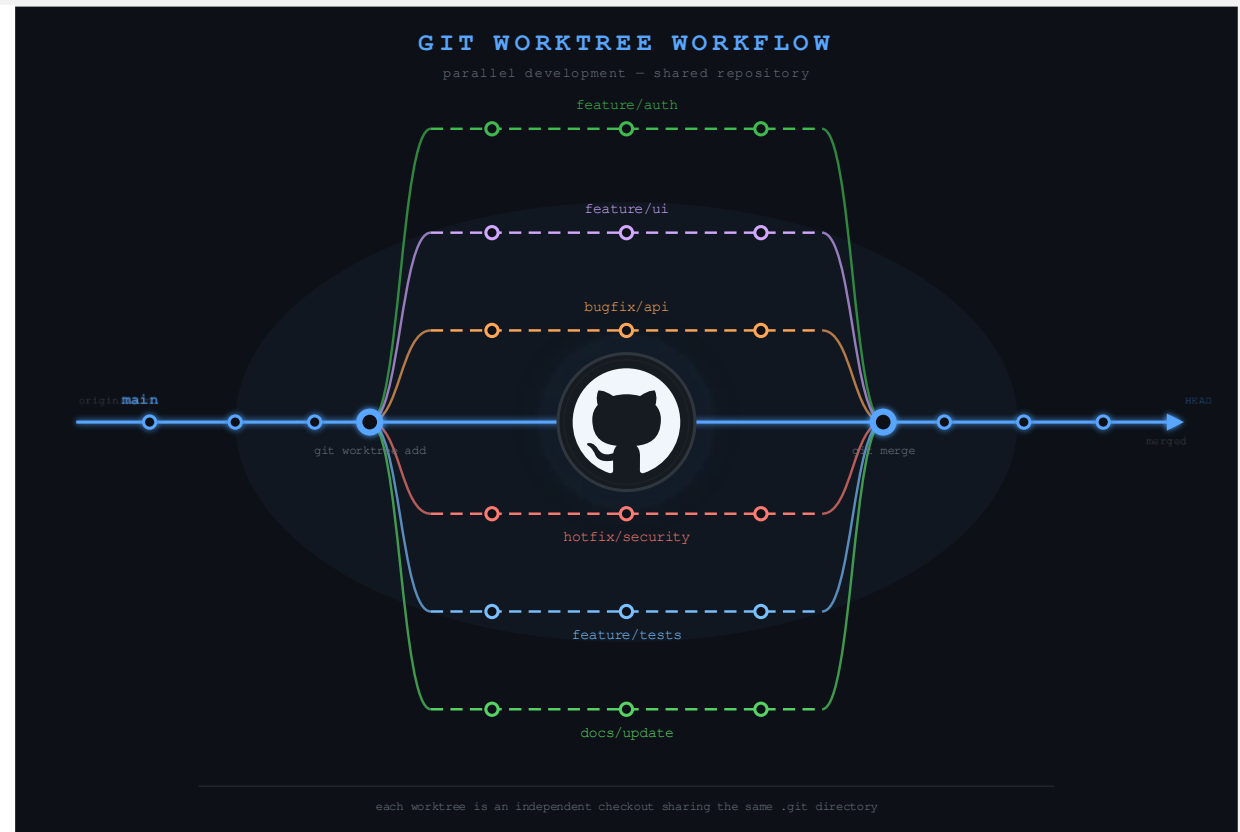
Issue #3: Scaling AI Development

From single-threaded work → parallel execution



After | GitHub App

- Multiple worktrees created automatically
- Tasks handled independently
- Copilot + agents work in parallel
- Faster turnaround.



Scaling AI Development

Let's take this a step further with the GitHub App

Factoring in Cost

Frontier models are expensive

Managing Cost & Token Usage

AI scale introduces a new constraint: cost

Usage-Based Cost

- AI requests consume tokens
- More context = higher cost
- Costs scale quickly

What Drives Cost

- Large prompts / context
- Repeated requests
- Overloaded memory
- Wrong model for the job

What is the cost

- Rising spend
- Run out of tokens
- Hard to predict usage

How We Control It

- Keep instructions minimal
- Limit context strategically
- Choose models intentionally
- Use subagents to scope tasks
- Don't break the cache

Picking the right model for the job

Auto is our friend!

How does auto work?

- Auto model selection with task optimization combines two systems to provide high-quality results and better reliability.
- One system tracks real-time system health and availability.
- One system evaluates task complexity.
- Putting these together, auto model selection routes the task to the optimal model.

Why use auto?

- Matches your task to the model that can solve it most efficiently.
- Picks the model choice based on real-time system health and availability; reduced rate limiting, lower latency and errors.
- Improved cost efficiency by picking the model that can do the job instead of everything being done by an Opus model.

Auto Mode Selection

Picking a model that works best for your tasks



Efficiency

- All prompts are routed to the most cost-efficient model that can perform the task.
- Well suited for fast and straightforward tasks; such as implementing a skeleton from an existing pattern



Balance

- Considers cost, quality, and latency to determine which model to select.
- Best for everyday use for work tasks.
- Works well for developing a new feature that is based on existing patterns that and requires new business logic.



Intelligence

- Prompts are evaluated for which model would provide the highest quality response.
- This works well for architecture decisions or in-depth analysis to restructure a system design.

Orchestrate with the right models

GitHub Hydrafusion

What is it?

- HydraFusion is a model that chooses a workflow that balances performance, cost, and latency for each task.
- Delivers frontier intelligence through runtime orchestration.
- Creates a full execution plan, choosing from models across multiple providers to draft, critique and revise, or cascade to more powerful models to complete your task.

Why does this help?

- Auto solved the problem of which model to choose for a single task. In this mode, when a task is comprised of many subtasks of different complexity, we have one model that can handle all of them.
- Instead, Hydrafusion has Adaptive multi-model orchestration where we can delegate subtasks to the right-sized model from its model pool.
- The result is significant cost savings when using Hydrafusion.

GitHub Copilot & VS Code Product Team



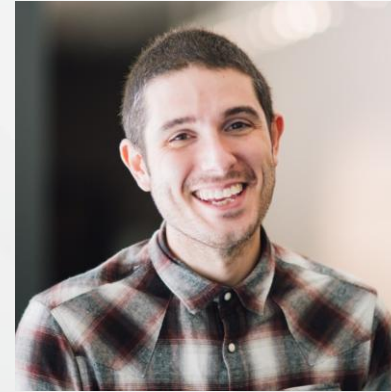
Pierce Boggan



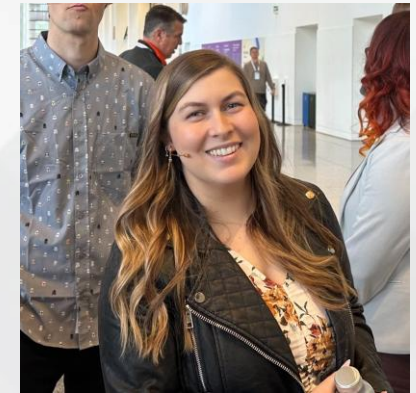
Burke Holland



Harald Kirschner



James Montemagno



Kayla Cinnamon

Integration Product Team



Harold Campos



Wagner Silveira



Divya Swarnkar



Thiago Almeida



Eldert Grootenboer

Integration Community



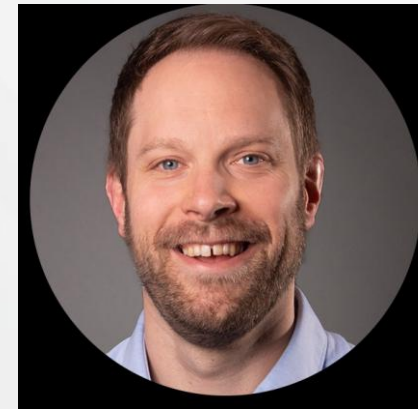
Michael Stephenson



Stephen W Thomas



Sandro Perreira



Sebastian Meyer



Steef-Jan Wiggers

Resources & Learning

GitHub Learn - <https://learn.github.com/courses>

GitHub Skills - <https://learn.github.com/skills>

VS Code AI Cheatsheet -

<https://code.visualstudio.com/docs/agents/reference/ai-features-cheat-sheet>

Awesome Copilot - <https://awesome-copilot.github.com/>

Brady Gaster Squad - <https://github.com/bradygaster/squad>

GitHub Copilot App - <https://github.com/features/ai/github-app>

MSLearn MCP - <https://learn.microsoft.com/en-us/training/support/mcp>

GitHub Usage Based Billing -

<https://docs.github.com/en/copilot/concepts/billing/usage-based-billing-for-organizations-and-enterprises>

30-50 % productivity gains for teams that use AI in SDLC by 2028 – [Gartner 2026¹](#)

70 % of enterprise software engineers will rely on AI coding agents for both synchronous and asynchronous development tasks by 2028 – [Gartner 2026¹](#)

65 % of engineering teams using agentic coding will treat IDEs as optional by 2027 – [Gartner 2026¹](#)

Final Thoughts

Leave you with two ideas and impressions...

01

Understanding

Understand the landscape of where GitHub Copilot fits for development in integration workflows, common issues and resolving those challenges.

02

Application

Take what we learn and use it immediately to improve your everyday development workflow with GitHub Copilot

Contact

Cameron McKay

Senior Manager

Cameron.McKay@mnp.ca

Wherever business takes you

[MNPdigital.ca](https://mnpdigital.ca)

